

## Módulo Maquetas HTML/ CSS

### CSS

**Objetivos:** Aprender a editar los estilos de una página web con código CSS diseñando los distintos elementos y clases. Entender el proceso para crear una sintaxis clara y precisa. Conocer los distintos recursos que podemos aplicar según las características del sitio que vamos a editar.

### CSS

CSS (cascading Style Sheets - en español Hojas de Estilo en Cascadas) es usado para darle estilo y diseño a las páginas web - por ejemplo, para cambiar la fuente de letra, color, tamaño y el espaciado de tu contenido; dividir en múltiples columnas, o agregar animaciones y otras propiedades decorativas.

#### Agregar CSS a un documento

Lo primero que se debe hacer es decirle al documento HTML que hay algunas reglas CSS que queremos que use. Hay tres formas diferentes de aplicar CSS a un documento HTML, sin embargo, por ahora, veremos la forma más habitual y útil de hacerlo: vincular el CSS desde el encabezado del documento.

Crea un archivo en la misma carpeta que tu documento HTML y guárdalo como *styles.css*. La extensión *.css* muestra que es un archivo CSS.

Para vincular *styles.css* a *index.html*, añade la siguiente línea en algún lugar dentro del `<head>` del documento HTML:

```
<link rel="stylesheet" href="styles.css">
```

Este elemento `<link>` le dice al navegador que hay una hoja de estilo con el atributo *rel* y la ubicación de esa hoja de estilo como el valor del atributo *href*.

Hay 3 grupos de declaraciones CSS que podemos implementar:

1. Formato de Texto: Familia tipográfica, color y tamaño de fuente, interlineado, negrita, itálicas, subrayados. (Heredadas)
2. Formato de Cajas: Ancho, alto, bordes, color e imágenes de fondo, márgenes sombra, rotación. (No Heredadas)
3. Ubicación de Elementos: Posicionamiento, flotaciones, mostrar/ocultar cajas.

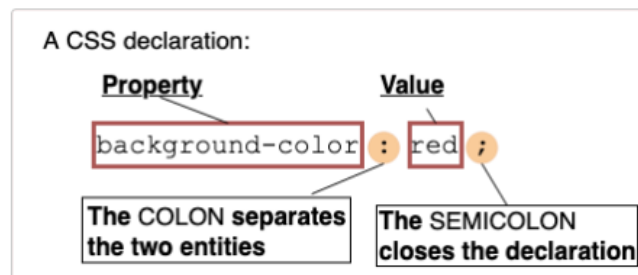
## Sintaxis

La meta básica del lenguaje CSS es permitir al motor del navegador pintar los elementos de la página con características específicas, como colores, posición o decoración. La sintaxis CSS refleja estas metas y estos son los bloques básicos de construcción.

- La **propiedad** que es un identificador, un *nombre* leíble por humanos, que define qué característica es considerada.
- El **valor** que describe como las características deben ser manejadas por el motor. Cada propiedad tiene un conjunto de valores válidos, definido por una gramática formal, así como un significado semántico, implementados por el motor del navegador.

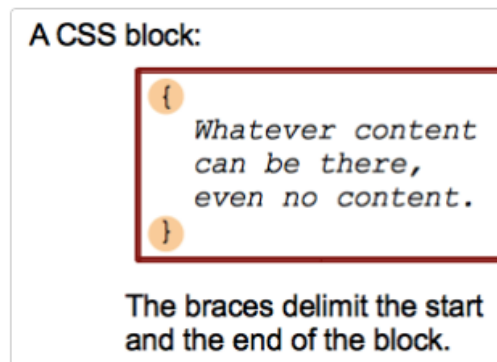
Configurando propiedades CSS a valores específicos es la función principal del lenguaje del CSS. Una propiedad y su valor son llamados una declaración, y cualquier motor de CSS calcula qué declaraciones aplican a cada uno de los elementos de una página para mostrarlos apropiadamente y estilizarlos.

Ambos, propiedades y valores, son sensibles a mayúsculas y minúsculas en CSS. El par se separa por dos puntos, ':', y los espacios en blanco antes, entre ellos y después, pero no necesariamente dentro de ellos, son ignorados.



Hay más de 100 propiedades diferentes en CSS y cerca de un número infinito de diferentes valores. No todos los pares de propiedades y valores son permitidos, cada propiedad define qué valores son válidos. Cuando un valor no es válido para una propiedad específica, la declaración es considerada inválida y es completamente ignorada por el motor de CSS.

Las declaraciones son agrupadas en bloques, que es una estructura delimitada por una llave de apertura, '{', y una de cierre, '}'. Los bloques en ocasiones pueden anidarse, por lo que las llaves de apertura y cierre deben de coincidir.



Esos bloques son naturalmente llamados bloques de declaraciones y las declaraciones dentro de ellos están separadas por un punto y coma, ';'. Un bloque de declaración puede estar vacío, que es contener una declaración nula. Los espacios en blanco alrededor de las declaraciones son ignorados. En cuanto a la última declaración de un bloque, esta no necesita terminar en un punto y coma, aunque es usualmente considerado una buena práctica porque previene el olvidar agregarlo cuando se extienda el bloque con otra declaración.

Ejemplo:

```
h1 {  
  color: red;  
  font-size: 5em;  
}
```

La regla se abre con un selector. Este selecciona el elemento HTML que vamos a diseñar. En este caso, diseñaremos encabezados de nivel uno (<h1>).

Luego tenemos un conjunto de llaves { }. Entre estas habrá una o más declaraciones, que tomarán la forma de pares de propiedad y valor. Cada par especifica cada una de las propiedades de los elementos seleccionados y el valor que queremos dar a esa propiedad.

Antes de los dos puntos, tenemos la propiedad; y después, el valor. Las propiedades CSS admiten diferentes valores, dependiendo de qué propiedad se esté especificando. En el ejemplo anterior, tenemos la propiedad color, que puede tomar varios valores de color. También tenemos la propiedad de font-size, que puede tomar varias unidades de tamaño como valor.

Una hoja de estilo CSS contendrá muchas de estas reglas, escritas una tras otra.

```
h1 {  
  color: red;  
  font-size: 5em;  
}  
  
p {  
  color: black;  
}
```

Una regla de CSS está formada por una parte llamada "selector" y otra parte llamada "declaración". La declaración indica "qué hay que hacer" y el selector indica "a quién hay que hacérselo". Por lo tanto, los selectores son imprescindibles para aplicar de forma correcta los estilos CSS en una página. A un mismo elemento HTML se le pueden asignar infinitas reglas CSS y cada regla CSS puede aplicarse a un número infinito de elementos. En otras palabras, una misma regla puede aplicarse sobre varios selectores y un mismo selector se puede utilizar en varias reglas.

## Selectores básicos

### Selector universal

Se utiliza para seleccionar todos los elementos de la página. El siguiente ejemplo elimina el margen y el relleno de todos los elementos HTML (por ahora no es importante fijarse en la parte de la declaración de la regla CSS): \* { margin: 0; padding: 0; } El selector universal se indica mediante un asterisco (\*). A pesar de su sencillez, no se utiliza habitualmente, ya que es difícil que un mismo estilo se pueda aplicar a todos los elementos de una página. No obstante, sí que se suele combinar con otros selectores y además, forma parte de algunos hacks muy utilizados.

## Selector de tipo o etiqueta

Selecciona todos los elementos de la página cuya etiqueta HTML coincide con el valor del selector. El siguiente ejemplo selecciona todos los párrafos de la página:

```
p {  
  ...  
}
```

Para utilizar este selector, solamente es necesario indicar el nombre de una etiqueta HTML (sin los caracteres < y >) correspondiente a los elementos que se quieren seleccionar. El siguiente ejemplo aplica diferentes estilos a los titulares y a los párrafos de una página HTML:

```
h1 {  
  color: red;  
}  
h2 {  
  color: blue;  
}  
p {  
  color: black;  
}
```

Si se quiere aplicar los mismos estilos a dos etiquetas diferentes, se pueden encadenar los selectores. En el siguiente ejemplo, los títulos de sección h1, h2 y h3 comparten los mismos estilos:

```
h1 {  
  color: #8A8E27;  
  font-weight: normal;  
  font-family: Arial, Helvetica, sans-serif;  
}  
h2 {  
  color: #8A8E27;  
  font-weight: normal;  
  font-family: Arial, Helvetica, sans-serif;  
}  
h3 {  
  color: #8A8E27;  
  font-weight: normal;  
  font-family: Arial, Helvetica, sans-serif;  
}
```

En este caso, CSS permite agrupar todas las reglas individuales en una sola regla con un selector múltiple. Para ello, se incluyen todos los selectores separados por una coma (,) y el resultado es que la siguiente regla CSS es equivalente a las tres reglas anteriores:

```
h1, h2, h3 {  
  color: #8A8E27;  
  font-weight: normal;  
  font-family: Arial, Helvetica, sans-serif;  
}
```

En las hojas de estilo complejas, es habitual agrupar las propiedades comunes de varios elementos en una única regla CSS y posteriormente definir las propiedades específicas de esos mismos elementos. El siguiente ejemplo establece en primer lugar las propiedades comunes de los títulos de sección (color y tipo de letra) y a continuación, establece el tamaño de letra de cada uno de ellos:

```
h1, h2, h3 {  
  color: #8A8E27;  
  font-weight: normal;  
  font-family: Arial, Helvetica, sans-serif;  
}  
h1 { font-size: 2em; }  
h2 { font-size: 1.5em; }  
h3 { font-size: 1.2em; }
```

## Selector descendente

Selecciona los elementos que se encuentran dentro de otros elementos. Un elemento es descendiente de otro cuando se encuentra entre las etiquetas de apertura y de cierre del otro elemento. El selector del siguiente ejemplo selecciona todos los elementos de la página que se encuentren dentro de un elemento:

```
p span { color: red; }
```

Si el código HTML de la página es el siguiente:

```
<p>
  ...
  <span>texto1</span>
  ...
  <a href="">...<span>texto2</span></a>
  ...
</p>
```

El selector `p span` selecciona tanto `texto1` como `texto2`. El motivo es que en el selector descendente, un elemento no tiene que ser "hijo directo" de otro. La única condición es que un elemento debe estar dentro de otro elemento, sin importar lo profundo que se encuentre. Al resto de elementos de la página que no están dentro de un elemento, no se les aplica la regla CSS anterior. Los selectores descendentes permiten mejorar la precisión del selector de tipo o etiqueta. Así, utilizando el selector descendente es posible aplicar diferentes estilos a los elementos del mismo tipo. El siguiente ejemplo amplía el anterior y muestra de color azul todo el texto de los contenidos dentro de un:

```
p span { color: red; }
h1 span { color: blue; }
```

Con las reglas CSS anteriores:

- Los elementos que se encuentran dentro de un elemento se muestran de color rojo.
- Los elementos que se encuentran dentro de un elemento se muestran de color azul.
- El resto de elementos de la página, se muestran con el color por defecto aplicado por el navegador.

La sintaxis formal del selector descendente se muestra a continuación:

elemento1 elemento2 elemento3 ... elementoN

Los selectores descendentes siempre están formados por dos o más partes separadas entre sí por espacios en blanco. La última parte indica el elemento sobre el que se aplican los estilos y todas las partes anteriores indican el lugar en el que se tiene que encontrar ese elemento para que los estilos se apliquen realmente.

En el siguiente ejemplo, el selector descendente se compone de cuatro partes:

```
p a span em { text-decoration: underline; }
```

Los estilos de la regla anterior se aplican a los elementos de tipo que se encuentren dentro de elementos de tipo `p`, que a su vez se encuentren dentro de elementos de tipo `a` que se encuentren dentro de elementos de tipo `span`. No debe confundirse el selector descendente con la combinación de selectores:

```
/* El estilo se aplica a todos los elementos "p", "a", "span" y "em" */
```

```
p, a, span, em { text-decoration: underline; }
```

```
/* El estilo se aplica sólo a los elementos "em" que se encuentran dentro de "p a span" */
```

```
p a span em { text-decoration: underline; }
```

Si se emplea el selector descendente combinado con el selector universal, se puede restringir el alcance de un selector descendente. El siguiente ejemplo, muestra los dos enlaces de color rojo:

```
p a { color: red; }

<p><a href="#">Enlace</a></p>
<p><span><a href="#">Enlace</a></span></p>
```

Sin embargo, en el siguiente ejemplo solamente el segundo enlace se muestra de color rojo:

```
p * a { color: red; }

<p><a href="#">Enlace</a></p>
<p><span><a href="#">Enlace</a></span></p>
```

La razón es que el selector `p * a` se traduce como todos los elementos de tipo que se encuentren dentro de cualquier elemento que, a su vez, se encuentre dentro de un elemento de tipo `p`. Como el primer elemento se encuentra directamente bajo un elemento `p`, no se cumple la condición del selector `p * a`.

## Selector de clase

Si se considera el siguiente código HTML de ejemplo:

```
<body>
  <p>Lorem ipsum dolor sit amet...</p>
  <p>Nunc sed lacus et est adipiscing accumsan...</p>
  <p>Class aptent taciti sociosqu ad litora...</p>
</body>
```

¿Cómo se pueden aplicar estilos CSS sólo al primer párrafo? El selector universal (\*) no se puede utilizar porque selecciona todos los elementos de la página. El selector de tipo o etiqueta (p) tampoco se puede utilizar porque seleccionaría todos los párrafos. Por último, el selector descendente (body p) tampoco se puede utilizar porque todos los párrafos se encuentran en el mismo sitio. Una de las soluciones más sencillas para aplicar estilos a un solo elemento de la página consiste en utilizar el atributo class de HTML sobre ese elemento para indicar directamente la regla CSS que se le debe aplicar:

```
<body>
  <p class="destacado">Lorem ipsum dolor sit amet...</p>
  <p>Nunc sed lacus et est adipiscing accumsan...</p>
  <p>Class aptent taciti sociosqu ad litora...</p>
</body>
```

A continuación, se crea en el archivo CSS una nueva regla llamada destacado con todos los estilos que se van a aplicar al elemento. Para que el navegador no confunda este selector con los otros tipos de selectores, se prefija el valor del atributo class con un punto (.) tal y como muestra el siguiente ejemplo:

```
.destacado { color: red; }
```

El selector .destacado se interpreta como "cualquier elemento de la página cuyo atributo class sea igual a destacado", por lo que solamente el primer párrafo cumple esa condición. Este tipo de selectores se llaman selectores de clase y son los más utilizados junto con los selectores de ID que se verán a continuación. La principal característica de este selector es que

en una misma página HTML varios elementos diferentes pueden utilizar el mismo valor en el atributo class:

```
<body>
  <p class="destacado">Lorem ipsum dolor sit amet...</p>
  <p>Nunc sed lacus et <a href="#" class="destacado">est adipiscing</a>
  accumsan...</p>
  <p>Class aptent taciti <em class="destacado">sociosqu ad</em> litora...</p>
</body>
```

Los selectores de clase son imprescindibles para diseñar páginas web complejas, ya que permiten disponer de una precisión total al seleccionar los elementos. Además, estos selectores permiten reutilizar los mismos estilos para varios elementos diferentes.

A continuación se muestra otro ejemplo de selectores de clase:

```
.aviso {
  padding: 0.5em;
  border: 1px solid #98be10;
  background: #f6feda;
}
.error {
  color: #930;
  font-weight: bold;
}
<span class="error">...</span>
<div class="aviso">...</div>
```

El elemento tiene un atributo class="error", por lo que se le aplican las reglas CSS indicadas por el selector .error. Por su parte, el elemento tiene un atributo class="aviso", por lo que su estilo es el que define las reglas CSS del selector .aviso.

## Selectores ID

En ocasiones, es necesario aplicar estilos CSS a un único elemento de la página. Aunque puede utilizarse un selector de clase para aplicar estilos a un único elemento, existe otro selector más eficiente en este caso.

El selector de ID permite seleccionar un elemento de la página a través del valor de su atributo id. Este tipo de selectores sólo seleccionan un elemento de la página porque el valor del atributo id no se puede repetir en dos elementos diferentes de una misma página.

La sintaxis de los selectores de ID es muy parecida a la de los selectores de clase, salvo que se utiliza el símbolo de la almohadilla (#) en vez del punto (.) como prefijo del nombre de la regla CSS:

```
#destacado { color: red; }
<p>Primer párrafo</p>
<p id="destacado">Segundo párrafo</p>
<p>Tercer párrafo</p>
```

En el ejemplo anterior, el selector #destacado solamente selecciona el segundo párrafo (cuyo atributo id es igual a destacado).

La principal diferencia entre este tipo de selector y el selector de clase tiene que ver con HTML y no con CSS. Como se sabe, en una misma página, el valor del atributo id debe ser único, de forma que dos elementos diferentes no pueden tener el mismo valor de id. Sin embargo, el atributo class no es obligatorio que sea único, de forma que muchos elementos HTML diferentes pueden compartir el mismo valor para su atributo class.

De esta forma, la recomendación general es la de utilizar el selector de ID cuando se quiere aplicar un estilo a un solo elemento específico de la página y utilizar el selector de clase cuando se quiere aplicar un estilo a varios elementos diferentes de la página HTML.

## Herencia: concepto Padre e Hijo

A la hora de crear las diferentes clases en el CSS, sirve para agregar atributos específicos a “hijos” sin alterar los atributos del “padre”, es decir, siguiendo el ejemplo a continuación: todos los elementos tipo párrafos son negros, pero si existe un párrafo que tenga como etiqueta string, ese párrafo será rojo, ya que el elemento strong es un hijo del elemento párrafo.

Selector HIJO  
↓  
Selector PADRE → **p strong {**  
**color: red;**  
**}**

Es decir, TODOS los STRONG que tengan como PADRE a P, tendrán texto **ROJO**

En HTML sería:

```
<p> Lorem ipsum dolor sit amet, consectetur adipiscing elit. Integer id
aliquam urna. <strong> Donec vitae luctus risus, quis sodales dolor. Nunc
ac facilisis nulla. In nisi massa, porttitor et eleifend ut, molestie vitae justo.
Donec a dignissim mauris</strong>. Nullam mattis bibendum mauris, ut ultrices
turpis auctor a.
</p>
```

## Cascada

Define una ruta que deben cumplir los elementos dentro del HTML. Se trata de una lista de elementos (ya sea por etiqueta, ID o clase) separados por espacios. Si algún elemento cumple esta “ruta” le aplica el formato CSS al último elemento de la lista (el que está justo antes de la apertura de llaves). Se usa para no plagar el HTML de atributos CLASS e ID.

Si tenemos dos listas distintas (un **<ol>** y un **<ul>**) necesitamos que los **<li>** de cada lista sean de distintos colores. Podemos aplicarle un class a cada list-item (pero esto nos obliga a copiar y pegar el class tantas veces como ítems tenga cada lista) o hacer una cascada discriminando los **<li>** dentro de cada lista.

Si se hace con class, hay que repetir el class por cada ítem del mismo tipo:

HTML:

```
<ul>
  <li class="rojo">Item R1</li>
  <li class="rojo">Item R2</li>
  <li class="rojo">Item R3</li>
  <li class="rojo">Item R4</li>
</ul>
<ol>
  <li class="azul">Item A1</li>
  <li class="azul">Item A2</li>
</ol>
```

CSS:

```
.rojo{ color: red; }
.azul{ color: blue; }
```

Si se hace con cascada todo <li> dentro de un <ul>, color rojo. Y todo <li> dentro de un <ol>, color azul:

HTML:

```
<ul>
  <li>Item R1</li>
  <li>Item R2</li>
  <li>Item R3</li>
  <li>Item R4</li>
</ul>
<ol>
  <li>Item A1</li>
  <li>Item A2</li>
</ol>
```

CSS:

```
ul li{ color: red; }
ol li{ color: blue; }
```

## Precedencia de declaraciones

Cuando reglas distintas apuntan al mismo objeto:

*Si son propiedades distintas se suman (se combinan).*

*Si tienen alguna propiedad repetida, solo una queda.*

Esto es lo que se llama **precedencia**.

- Las reglas por etiqueta tienen menos precedencia (porque son genéricas).
- La declaración de Class sobrescribe las de etiqueta (porque tenés que darle a mano esa clase).
- La declaración de ID pisa cualquier otra regla (porque el ID es único, por lo cual es más específico).

## !Important;

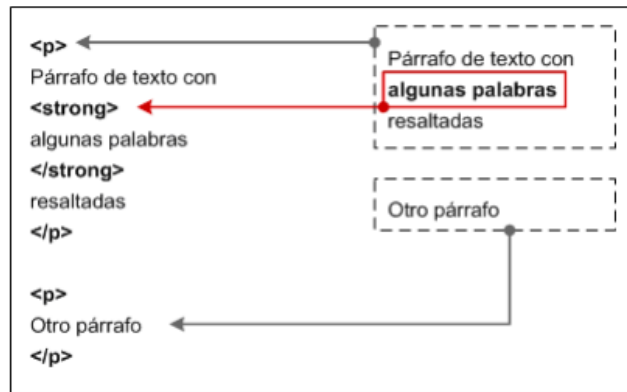
En un CSS extenso es común que algunas reglas CSS “choquen”. La declaración **!important;** corta la precedencia. Se escribe después del valor de la propiedad CSS, que se quiere convertir en la más importante. Sí, es un **!important;** por cada valor a pisar.

**Si se necesitan más de 5 !important; en todo el CSS, algo se está haciendo mal.**

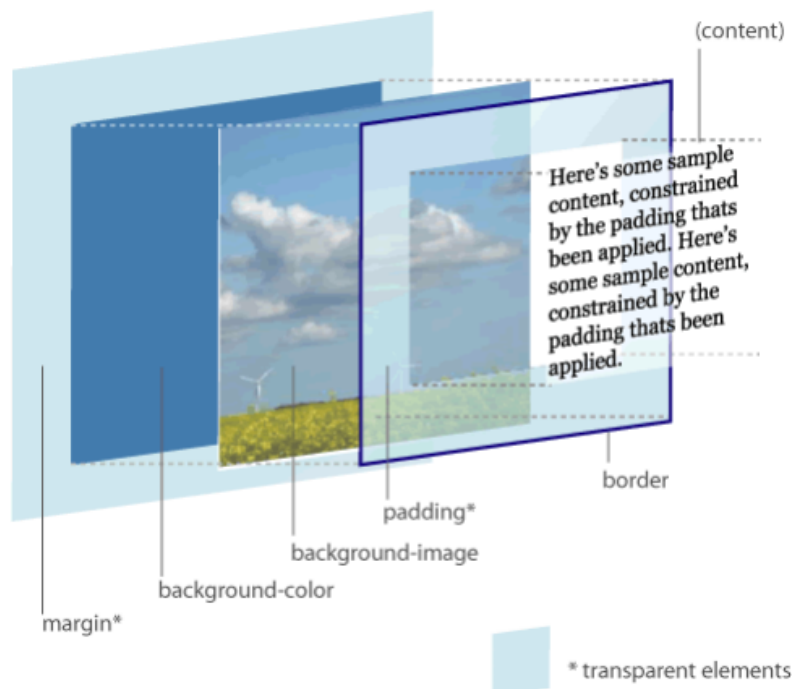
## Box Model

El modelo de cajas o "box model" es seguramente la característica más importante del lenguaje de hojas de estilos CSS, ya que condiciona el diseño de todas las páginas web. El "box model" es el comportamiento de CSS que hace que todos los elementos incluidos en una página HTML se representen mediante cajas rectangulares. CSS permite controlar el aspecto de todas las cajas. El diseño de cualquier página XHTML está compuesto por cajas rectangulares. CSS permite definir la altura y anchura de cada caja, el margen existente entre cajas y el espacio de relleno interior que muestra cada caja. Además, CSS permite controlar la forma en la que se visualizan las cajas: se pueden ocultar, desplazar respecto de su posición original y fijarlas en una posición específica dentro del documento. Como la mayoría de cajas de las páginas web no muestran ningún color de fondo ni ningún borde, no son visibles a simple vista.

Las cajas de una página se crean automáticamente. Cada vez que se inserta una etiqueta o elemento en la página, se crea una nueva caja rectangular que encierra los contenidos del elemento. El siguiente esquema muestra la creación automática de cajas por parte de HTML para cada elemento definido en el código HTML de la página:



Cada una de las cajas está formada por seis partes, tal y como se muestra en la siguiente imagen tridimensional:



Las partes que componen cada caja y su orden de visualización desde el punto de vista del usuario son las siguientes:

- Contenido (**content**): se trata del contenido HTML del elemento (las palabras de un párrafo, una imagen, el texto de una lista de elementos, etc.)
- Relleno (**padding**): espacio libre opcional entre el contenido y el borde que lo encierra.
- Borde (**border**): línea que encierra completamente el contenido y su relleno.
- Imagen de fondo (background image): imagen que se muestra por detrás del contenido y el espacio de relleno.

- Color de fondo (**background color**): color que se muestra por detrás del contenido y el espacio de relleno.
- Margen (**margin**): espacio libre entre la caja y las posibles cajas adyacentes.

El relleno y el margen son transparentes, por lo que en el espacio ocupado por el relleno se muestra el color o imagen de fondo (si están definidos) y en el espacio ocupado por el margen se muestra el color o imagen de fondo de su elemento padre (si están definidos). Si ningún elemento padre tiene definido un color o imagen de fondo, se muestra el color o imagen de fondo de la propia página (si están definidos). Si una caja define tanto un color como una imagen de fondo, la imagen tiene más prioridad y es la que se visualiza. No obstante, si la imagen de fondo no cubre totalmente la caja del elemento o si la imagen tiene zonas transparentes, también se visualiza el color de fondo. Combinando imágenes transparentes y colores de fondo se pueden lograr efectos gráficos muy interesantes.

## Ancho

La propiedad CSS que controla la anchura de los elementos se denomina **width**.

La propiedad width no admite valores negativos y los valores en porcentaje se calculan a partir de la anchura de su elemento padre. El valor inherit indica que la anchura del elemento se hereda de su elemento padre. El valor auto, que es el que se utiliza si no se establece de forma explícita un valor a esta propiedad, indica que el navegador debe calcular automáticamente la anchura del elemento, teniendo en cuenta sus contenidos y el sitio disponible en la página.

El siguiente ejemplo establece el valor de la anchura del elemento <div> lateral:

```
#lateral { width: 200px; }  
  
<div id="lateral">  
...  
</div>
```

## Altura

La propiedad CSS que controla la altura de los elementos se denomina **height**.

Al igual que sucede con width, la propiedad height no admite valores negativos. Si se indica un porcentaje, se toma como referencia la altura del elemento padre. Si el elemento padre no tiene una altura definida explícitamente, se asigna el valor auto a la altura. El valor inherit indica que la altura del elemento se hereda de su elemento padre. El valor auto, que es el que se utiliza si no se establece de forma explícita un valor a esta propiedad, indica que el navegador debe calcular automáticamente la altura del elemento, teniendo en cuenta sus contenidos y el sitio disponible en la página.

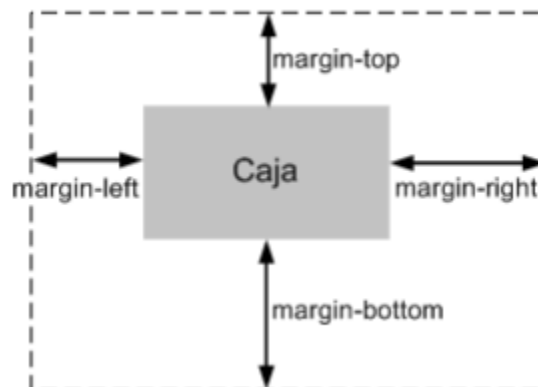
El siguiente ejemplo establece el valor de la altura del elemento <div> de cabecera:

```
#cabecera { height: 60px; }  
<div id="cabecera">  
...  
</div>
```

## Margen

CSS define cuatro propiedades para controlar cada uno de los márgenes horizontales y verticales de un elemento.

Cada una de las propiedades establece la separación entre el borde lateral de la caja y el resto de cajas adyacentes:



Las unidades más utilizadas para indicar los márgenes de un elemento son los píxeles (cuando se requiere una precisión total), los em (para hacer diseños que mantengan las proporciones) y los porcentajes (para hacer diseños líquidos o fluidos).

El siguiente ejemplo añade un margen izquierdo al segundo párrafo:

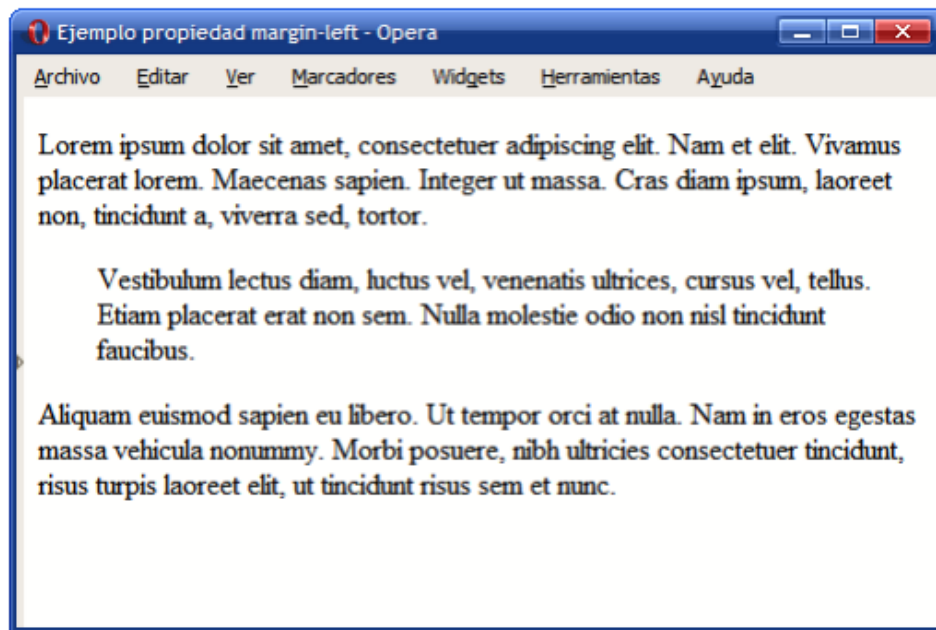
```
destacado {
  margin-left: 2em;
}

<p>Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nam et elit. Vivamus placerat lorem. Maecenas sapien. Integer ut massa. Cras diam ipsum, laoreet non, tincidunt a, viverra sed, tortor.</p>

<p class="destacado">Vestibulum lectus diam, luctus vel, venenatis ultrices, cursus vel, tellus. Etiam placerat erat non sem. Nulla molestie odio non nisl tincidunt faucibus.</p>

<p>Aliquam euismod sapien eu libero. Ut tempor orci at nulla. Nam in eros egestas massa vehicula nonummy. Morbi posuere, nibh ultricies consectetur tincidunt, risus turpis laoreet elit, ut tincidunt risus sem et nunc.</p>
```

A continuación se muestra el aspecto del ejemplo anterior en cualquier navegador:



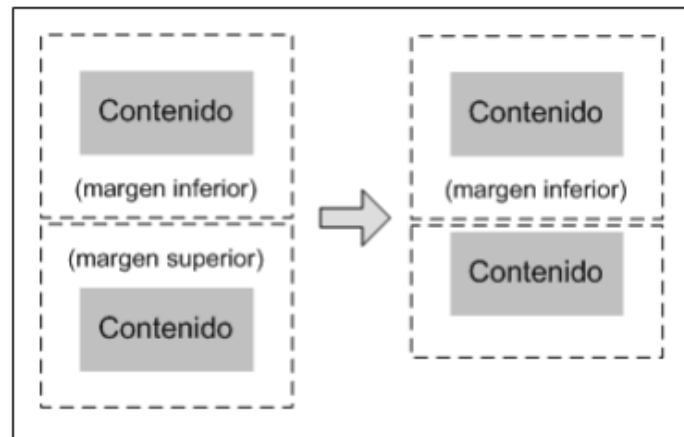
Los márgenes verticales (`margin-top` y `margin-bottom`) sólo se pueden aplicar a los elementos de bloque y las imágenes, mientras que los márgenes laterales (`margin-left` y `margin-right`) se pueden aplicar a cualquier elemento.

La propiedad que permite definir de forma simultánea los cuatro márgenes se denomina **margin**.

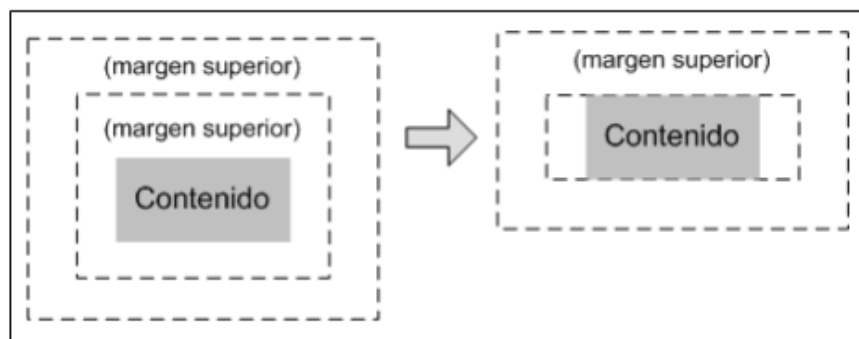
La notación {1, 4} de la definición anterior significa que la propiedad margin admite entre uno y cuatro valores, con el siguiente significado:

- Si solo se indica un valor, todos los márgenes tienen ese valor.
- Si se indican dos valores, el primero se asigna al margen superior e inferior y el segundo se asigna a los márgenes izquierdo y derecho.
- Si se indican tres valores, el primero se asigna al margen superior, el tercero se asigna al margen inferior y el segundo valor se asigna los márgenes izquierdo y derecho.
- Si se indican los cuatro valores, el orden de asignación es: margen superior, margen derecho, margen inferior y margen izquierdo.

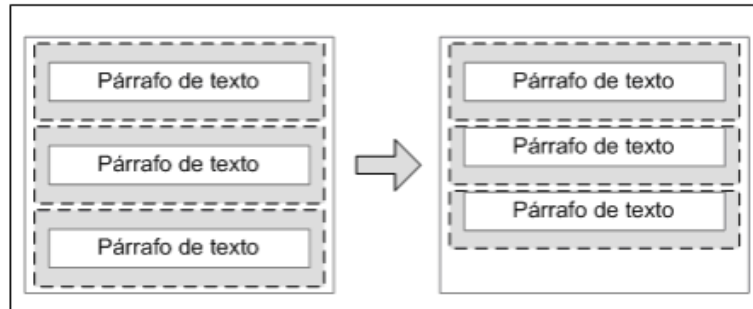
El comportamiento de los márgenes verticales es más complejo de lo que se puede imaginar. Cuando se juntan dos o más márgenes verticales, se fusionan de forma automática y la altura del nuevo margen será igual a la altura del margen más alto de los que se han fusionado.



De la misma forma, si un elemento está contenido dentro de otro elemento, sus márgenes verticales se fusionan y resultan en un nuevo margen de la misma altura que el mayor margen de los que se han fusionado:



Aunque en principio puede parecer un comportamiento extraño, la razón por la que se propuso este mecanismo de fusión automática de márgenes verticales es el de dar uniformidad a las páginas web habituales. En una página con varios párrafos, si no se diera este comportamiento y se estableciera un determinado margen a todos los párrafos, el primer párrafo no mostraría un aspecto homogéneo respecto de los demás.

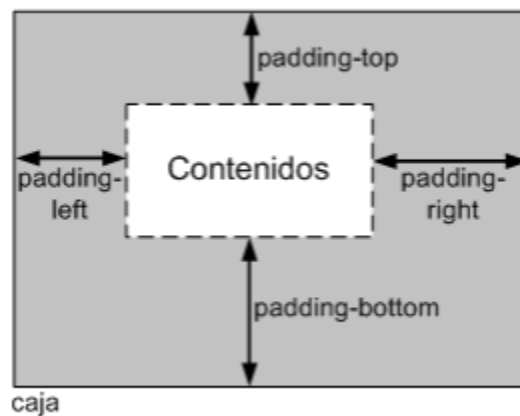


En el caso de un elemento que se encuentra en el interior de otro y sus márgenes se fusionan de forma automática, se puede evitar este comportamiento añadiendo un pequeño relleno (padding: 1px) o un borde (border: 1px solid transparent) al elemento contenedor.

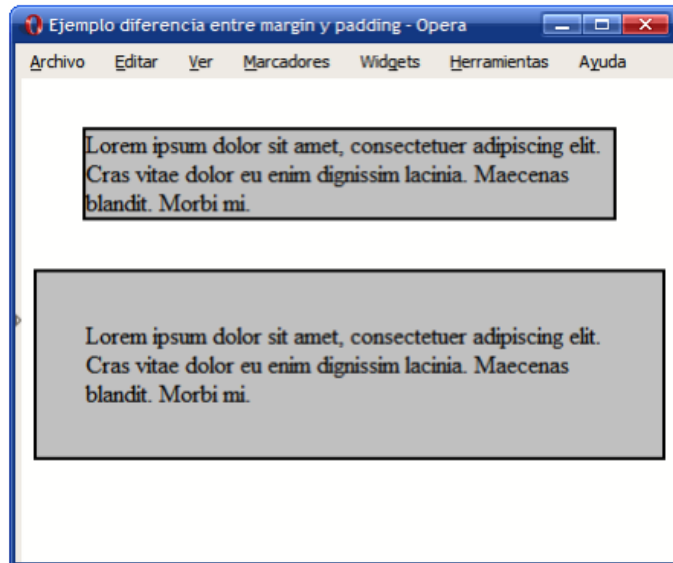
## Relleno

CSS define cuatro propiedades para controlar cada uno de los espacios de relleno horizontales y verticales de un elemento.

Cada una de las propiedades establece la separación entre el lateral de los contenidos y el borde lateral de la caja:



La siguiente imagen muestra la diferencia entre el margen y el relleno de los elementos:



El código HTML y CSS del ejemplo se muestra a continuación:

```
.margen {
  margin-top: 2em; margin-right: 2em; margin-bottom: 2em; margin-left: 2em;
}

.relleno {
  padding-top: 2em; padding-right: 2em; padding-bottom: 2em; padding-left: 2em;
}

<p class="margen">Lorem ipsum dolor sit amet, consectetur adipiscing elit.
Cras vitae dolor eu enim dignissim lacinia. Maecenas blandit. Morbi mi.</p>

<p class="relleno">Lorem ipsum dolor sit amet, consectetur adipiscing elit.
Cras vitae dolor eu enim dignissim lacinia. Maecenas blandit. Morbi mi.</p>
```

## Bordes

Entre el **Margin** y el **Padding** encontramos al **Border** el cual requiere de 3 valores (**width**, **style** y **color**) para poder ser visualizado correctamente.

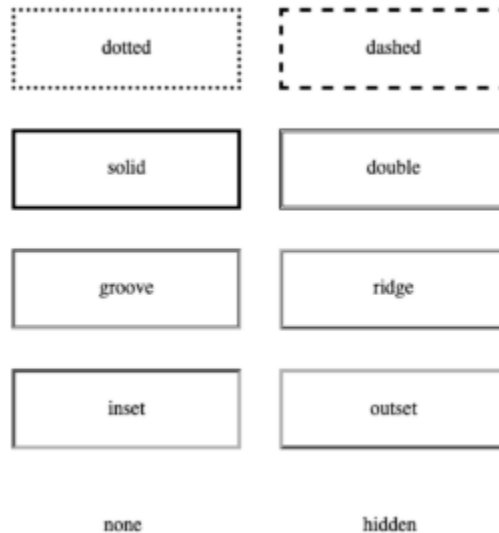
```
#caja {
  border: 20px solid #3EC483;
}
```

Si queremos establecer valores más específicos a los lados del border, cambiar el estilo o color solo debemos escribir:

`border-(top, bottom, left o right)-(width, style o color)`

`border-top-style` o `border-left-width`

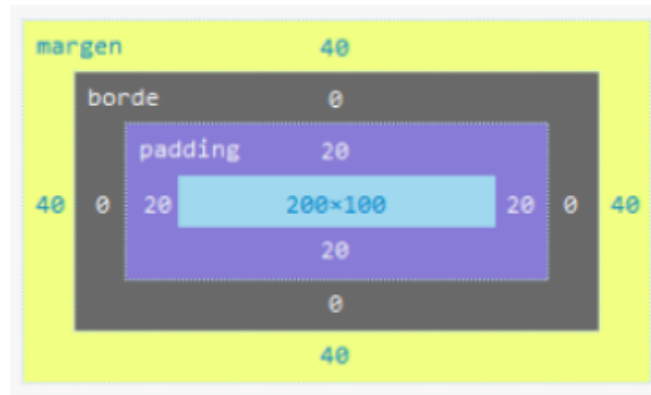
## Tipos de Borders (border-style)



Cuando deseamos hacer que un borde sea redondeado utilizamos `border-radius`. Esta propiedad modifica la forma en la que se muestran las esquinas de nuestra caja, *border-radius* acepta valores en píxeles y en porcentajes, además podemos establecer valores para cada borde por separado, muy parecido a lo que vimos anteriormente.

```
#caja {
  border: 20px solid #3EC483;
  border-radius: 25px;
}
```

¿Cómo se vería con las tres propiedades?



## Display

Se encarga de **definir cómo se ve un elemento HTML**. Los dos comportamientos más importantes son:

- Pasar un elemento de bloque a elemento de línea.
- Pasar un elemento de línea a elemento de bloque.

Eso se hace con los valores *block* e *inline* respectivamente:

- *block*: Convierte el elemento en elemento de bloque
- *inline*: Convierte el elemento en elemento de línea

```
#caja {
  display: block;
}
```

```
#caja {
  display: inline;
}
```

### Display Inline

Los elementos de línea se muestran uno al lado del otro pero tienen un problema: **No entienden los márgenes ni *padding* de arriba y abajo**. Si bien agrandan el elemento, éste se superpone a lo que esté arriba y abajo del mismo. Para que entienda el *padding* y margen de arriba y abajo, se usa *display: inline-block*. Se trata de un elemento de línea (uno al lado del otro) respetando todas las reglas físicas de uno de bloque.

```
#caja {  
  display: inline-block;  
}
```

### Bug del *display: inline-block*

El *display inline-block* con ancho fijo pone las cosas una al lado de la otra (sí, lo dijimos recién). Pero si tenés los anchos milimétricamente calculados puede ser que el último aparezca abajo (y no al lado). En los elementos de línea (inline o inline-block) si entre los elementos hay espacio (sea uno o 500 enter, espacios o tab) **ese aire entre elementos se muestra como un espacio de barra espaciadora**. De ahí que no te entren y se muestre uno debajo de los demás.

### Quitar un elemento

El *display* tiene también un valor para quitar un elemento del layout *display: none*; Lo oculta y, además, lo quita (no ocupa su lugar).

```
#caja {  
  display: none;  
}
```

### Float

La flotación es **mover un elemento hacia la derecha o izquierda de su línea y todo lo que viene después se acomodará en el “hueco” que queda vacío**. Es una manera ‘old fashion’ de hacer una columna. Se usa la propiedad *float* que acepta dos valores:

- *left*: Corre la caja a la izquierda.
- *right*: Corre la caja a la derecha.

**IMPORTANTE:** Cuando un elemento flota, deja de pertenecer al flujo normal del HTML.

```
#caja {  
  float: left;  
}
```

```
#caja {  
  float: right;  
}
```

Todo elemento flotado, deja de “empujar” en alto a su contenedor. Si todos los elementos flotan, el contenedor colapsa su altura.

## Position

Es una propiedad CSS pensada para ubicar un elemento en una coordenada puntual. Puede ser en base al BODY o al elemento que contiene el objeto posicionado.

Se suele usar position (entre otras cosas) para:

- Superponer elementos (como las ventanas modales).
- Crear barras de herramientas fijas (como facebook).
- Crear publicidades que te sigan con el scroll.
- Hacer un menú con submenú adentro.

## Cómo ubicar un elemento.

1. En primer lugar hay que definir qué tipo de posición se quiere usar.
2. Luego hay que indicar las coordenadas en las que se posiciona ese objeto.
3. Se hace indicando al objeto un valor numérico para las propiedades
  - a. *top, bottom, left, right.*
4. No se puede indicar dos propiedades opuestas a la vez (un objeto no puede estar en top y bottom).

## ¿Qué position existen?

Hay cinco valores posibles para el position:

- **fixed:** Los bordes serán calculados en base al browser. Si el usuario hace scroll, el objeto sigue fijo.
- **absolute:** Los bordes serán cualquier elemento contenedor que tenga una position relative. Si no encuentra ninguno, usa el body. Si se hace scroll, el objeto posicionado se mueve.
- **relative:** Sólo delimita los límites de cualquier elemento hijo con position absolute.
- **static:** Se usa para quitar un position
- **Sticky:** Se usa para posicionar en función de la posición de desplazamiento del usuario. Un elemento *sticky* alterna entre *relative* y *static*, dependiendo de la posición de desplazamiento. Se posiciona en forma relativa hasta que se alcanza una posición de desplazamiento determinada en la ventana gráfica, luego se "pega" en su lugar (como posición: fija).

## Propiedad z-index

El z-index entra en juego cuando dos elementos que tienen *position* se superponen. Esta propiedad acepta como valor un número (sin ninguna unidad, ni px, ni cm, ni nada). **A valor más alto, se mostrará por encima de los demás elementos.**

Por defecto todos los objetos tienen z-index:1. Si dos objetos tienen el mismo valor de z-index y se superponen, el que fue creado después en el HTML se verá encima del otro.

## Position sticky (pegajoso)

Es un nuevo valor del *position* que **mezcla un elemento sin position con uno fixed.**

Al igual que los demás, usa coordenadas *top*, *bottom*, *left* y *right*, pero se comporta distinto:

- Mientras el objeto se encuentre en una ubicación con valor superior al indicado, se moverá junto con el scroll.
- Cuando el objeto esté por debajo de ese valor (por fuera de los bordes), quedará fijo, como si estuviese *fixed*.

Ideal, por ejemplo, para un menú con anclas, si se tiene demasiado scroll vertical.

# Flex Box

Flex Box es un método de diseño de página unidimensional para compaginar elementos en filas o columnas. Los elementos de contenido se ensanchan para rellenar el espacio adicional y se encogen para caber en espacios más pequeños.

El flexbox (más concretamente, CSS Flexible Box Layout o diseño de caja flexible CSS) funciona de manera más inteligente y dinámica: el diseño se adapta de forma flexible a la pantalla donde se muestra, siguiendo el concepto del diseño receptivo. El espacio se llena o los elementos se desplazan de modo que todo permanezca visible. Para lograrlo sin dismantelar el diseño por completo, flexbox funciona con **dos ejes**: el eje principal, que suele ser el horizontal, y el eje transversal o vertical. Mediante estos ejes, los elementos se organizan dentro de la caja y se distribuyen en relación unos con otros. Una vez hecho esto, CSS Flexbox no tiene más que garantizar que el espacio que hay alrededor de estos elementos se llene correctamente.

Flexbox se basa en un contenedor flexible (flex container), que a su vez contiene varios elementos flexibles (flex items). El contenedor otorga sus propiedades a los elementos, es decir: los elementos o flexboxes deben su flexibilidad al hecho de estar dentro del contenedor. Cada uno de los dos ejes sigue una dirección: por lo general, el eje principal va de izquierda a derecha, mientras que el eje transversal va de arriba a abajo. Flexbox se describe como un sistema unidimensional: los elementos se pueden organizar en filas o columnas, y nunca se pretende combinar ambas. Por ejemplo, si te decides por la disposición en filas (la estándar de esta tecnología), CSS Flexbox intentará organizar todos los elementos en una sola fila, aunque también es posible evitarlo y forzar un salto de línea.

## Crear y organizar los elementos

Antes de comenzar a organizar los elementos, primero debes crearlos. La forma de hacerlo es mediante HTML, como de costumbre:

```
<!DOCTYPE html>
<html>
<head>
</head>
<body>
<div class="flex-container">
  <div class="flex-item">Box 1</div>
  <div class="flex-item">Box 2</div>
  <div class="flex-item">Box 3</div>
</div>

</body>
</html>
```

Los tres elementos se mostrarían simplemente uno debajo del otro. Ahora, utilizamos CSS para distribuirlos en el eje principal:

```
<style>
.flex-container {
  display: flex;
  background-color: grey;
}
.flex-item {
  background-color: white;
  width: 200px;
  margin: 10px;
  text-align: center;
  line-height: 100px;
  font-size: 50px;
}
</style>
```



En esta hoja de estilo, se ha definido el contenedor. La propiedad *display: flex* habilita la flexbox. Los elementos se distribuyen de izquierda a derecha, ya que no lo hemos definido de otra manera. También es posible hacerlo mediante cinco opciones distintas, configurables con el comando *justify-content*:

- *flex-start*: justificado a la izquierda
- *flex-end*: justificado a la derecha
- *center*: justificado en el centro
- *space-around*: distribuye uniformemente el espacio alrededor de las cajas
- *space-between*: distribuye uniformemente el espacio entre las cajas

El resto del código hasta este punto solo es estético y no tiene nada que ver con las cajas flexibles en sí.

El modelo CSS Flexbox parte de la alineación horizontal, aunque también es posible organizar los elementos en una columna. Además, se puede invertir la dirección, de izquierda a derecha o de abajo a arriba. Para ello, se utiliza el comando *flex-direction*:

- *row*: de izquierda a derecha
- *row-reverse*: de derecha a izquierda
- *column*: de arriba abajo

- *column-reverse*: de abajo a arriba

El comando *justify-content: flex-end* no es igual que *flex-direction: row-reverse*. Mientras que en el primero se ajusta el último elemento al borde derecho, en el segundo, se cambia el orden completamente, lo que significa que el primer elemento del código aparece en el borde derecho.

## Agrupar elementos

Hasta ahora, hemos colocado las cajas flexibles de manera uniforme. No obstante, en diseño web, no siempre deseamos que todos los elementos se distribuyan por igual. Por ejemplo, a veces queremos que el texto se muestre de manera diferente a una imagen. Para lograrlo, podemos agrupar los elementos en el texto HTML:

```
<div class="flex-container">
  <div class="flex-item">Box 1</div>
  <div class="flex-item">
    <div class="flex-item">Box 2</div>
    <div class="flex-item">Box 3</div>
  </div>
</div>
```



**Desplazamiento vertical**

Con *justify-content*, los elementos se pueden distribuir con una cierta relación entre sí en el eje horizontal. En cambio, si quieres que el contenido se muestre de arriba a abajo (en el eje transversal), necesitarás el comando *align-items*. En este caso, también hay cinco opciones distintas:

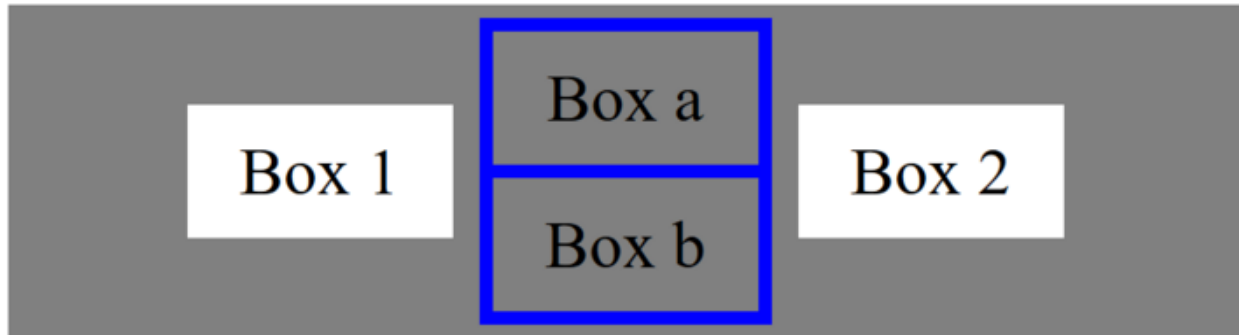
- *center*: los objetos se centran.
- *flex-start*: los objetos se ajustan al borde superior.
- *flex-end*: los objetos se ajustan al borde inferior.
- *stretch*: los objetos se muestran con el mismo tamaño.

- baseline: los objetos se organizan en la línea de base (según el contenido).

A primera vista, las opciones flex-start y baseline parecen ofrecer los mismos resultados, pero presentan una diferencia que se hace especialmente evidente cuando los objetos están anidados: mientras que flex-start solo vincula dos flexboxes que están en el mismo nivel jerárquico, baseline también tiene en cuenta el contenido que hay en las cajas.

Con el siguiente código, se pueden centrar tres objetos de diferentes tamaños uno al lado del otro:

```
<!DOCTYPE html>
<html>
<head>
<style>
.flex-container {
  display: flex;
  background-color: grey;
  justify-content: center;
  align-items: center;
}
.flex-item1 {
  background-color: white;
  width: 200px;
  margin: 10px;
  text-align: center;
  line-height: 100px;
  font-size: 50px;
}
.flex-item2 {
  background-color: blue;
  width: auto;
  margin: 10px;
  text-align: center;
  line-height: 100px;
  font-size: 50px;
}
.flex-item3 {
  background-color: grey;
  width: 200px;
  margin: 10px;
  text-align: center;
  line-height: 100px;
  font-size: 50px;
}
</style>
</head>
<body>
<div class="flex-container">
  <div class="flex-item1">Box 1</div>
  <div class="flex-item2">
    <div class="flex-item3">Box a</div>
    <div class="flex-item3">Box b</div>
  </div>
  <div class="flex-item1">Box 2</div>
</div>
</body>
</html>
```



La distribución vertical también puede modificarse mediante saltos de línea. Si colocaras muchos objetos en un contenedor, sencillamente se seguirían mostrando en formato horizontal, de modo que el usuario tendría que desplazarse y salir del área de la pantalla para verlos. Con flex-wrap, te aseguras de que los elementos se distribuyen de forma ordenada en varias líneas.

- nowrap: sin salto de línea
- wrap: salto de línea
- wrap-reverse: salto de línea (ordenado a la inversa)

```
.flex-container {
  display: flex;
  background-color: grey;
  justify-content: center;
  align-items: center;
  flex-wrap: wrap-reverse;
}
```

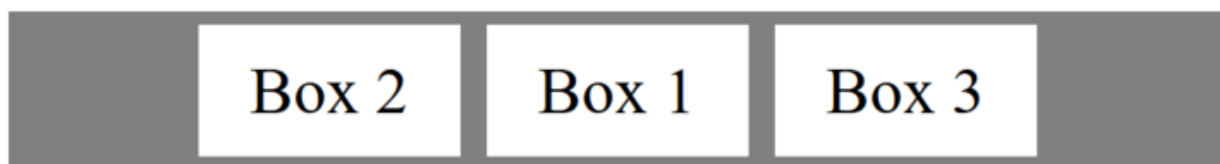
## Desplazar una única flexbox

Hasta ahora, la disposición siempre se ha aplicado a todo el contenedor, de manera que todas las cajas incluidas dentro del mismo siguen los comandos generales. Esto facilita el trabajo, pero, al mismo tiempo, también lo limita. Por este motivo, CSS Flexbox ofrece a los diseñadores web la oportunidad de definir excepciones. Para ello, se utiliza el comando **order**. El valor predeterminado (si el usuario no define otro) es 0. El valor añade el objeto a un grupo abstracto: todos los elementos que hemos creado hasta ahora pertenecen al mismo grupo, porque todos tienen el valor 0. Partiendo de este valor, ahora puedes mover cada objeto hacia adelante (-1) o hacia atrás (1).

Si dejas todos los elementos en valor 0 excepto uno al que le des el valor 1, este objeto se mostrará después del resto de elementos. El objeto también puede desplazarse asignándole otros valores (superiores o inferiores). Ten en cuenta que se trata de una **representación puramente visual**: la secuencia lógica de acuerdo con el documento HTML no cambia.

```
<!DOCTYPE html>
<html>
<head>
<style>
.flex-container {
  display: flex;
  background-color: grey;
  justify-content: center;
  align-items: center;
  flex-direction: row;
}
.flex-item1 {
  background-color: white;
  width: 200px;
  margin: 10px;
  text-align: center;
  line-height: 100px;
  font-size: 50px;
}
.flex-item2 {
  background-color: white;
  width: 200px;
  margin: 10px;
  text-align: center;
  line-height: 100px;
  font-size: 50px;
  order: -1;
}
</style>
</head>
<body>
<div class="flex-container">
  <div class="flex-item1">Box 1</div>
  <div class="flex-item2">Box 2 </div>
  <div class="flex-item1">Box 3</div>
</div>
</body>
</html>
```

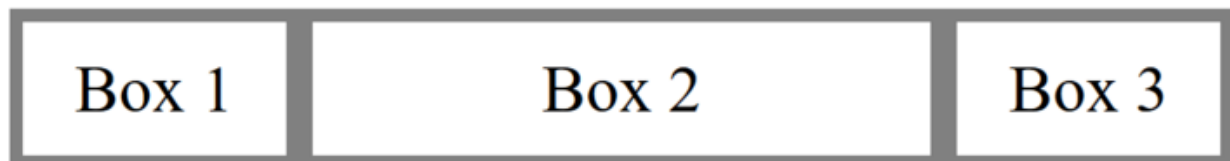
## Tamaño flexible de los elementos



De momento solo hemos hablado de la disposición flexible de los objetos. En todos los casos, las relaciones de tamaño se han establecido mediante el CSS clásico en el código de muestra. No obstante, las flexboxes también pueden adaptarse en tamaño. Para ello, se utiliza el comando **flex**. De forma parecida al orden, los elementos pueden dividirse en grupos; cuanto mayor es el valor, más espacio ocupa el elemento.

```
.flex-container {
  display: flex;
  background-color: grey;
  justify-content: center;
  align-items: center;
  flex-direction: row;
}
.flex-item1 {
  background-color: white;
  width: 200px;
  margin: 10px;
  text-align: center;
  line-height: 100px;
  font-size: 50px;
}
.flex-item2 {
  background-color: white;
  width: 200px;
  margin: 10px;
  text-align: center;
  line-height: 100px;
  font-size: 50px;
  flex: 1;
}
```

De hecho, *flex* es una forma abreviada del comando, que contiene tres instrucciones: *flex-grow*, *flex-shrink* y *flex-basis*. Así, puedes introducir los valores individuales directamente en forma abreviada (*flex: <flex-grow> <flex-shrink> <flex-basis> / flex: 1 1 20em*) o determinar un solo valor y dejar que CSS se encargue de la configuración.



Material fuente:

<https://developer.mozilla.org/es/docs/Learn/CSS>

<http://www.librosweb.es/css>

<https://www.coderhouse.com/categorias/programacion>

<https://www.ionos.es/digitalguide/paginas-web/creacion-de-paginas-web/css-flexbox/>